

Guía Pedagógica

Searching Latent Program Spaces

El paper de la **Latent Program Network (LPN)**, explicado paso a paso

Basado en: Macfarlane, M. V. & Bonnet, C. (2025).
Searching Latent Program Spaces. NeurIPS 2025. arXiv:2411.08706

Cómo está pensada esta guía:

Vamos a recorrer el paper sección por sección, como en una clase particular. Cada concepto nuevo se explica desde cero, cada ecuación se “traduce” a palabras simples, y al final de cada sección vas a encontrar preguntas diseñadas para que pienses *con tus propias palabras*, no solo para que memorices. No hace falta que sepas nada de síntesis de programas o de autoencoders variacionales antes de empezar: lo iremos construyendo juntos.

Índice

1	Antes de empezar: el mapa del paper	2
2	Introducción: el problema y la motivación	3
2.1	El objetivo de fondo: inteligencia general	3
2.2	Dos familias de métodos: inductivo vs. transductivo	3
3	Fundamentos formales: ¿qué es sintetizar un programa?	5
3.1	Especificación de una tarea	5
3.2	El verdadero objetivo: generalizar a un input nuevo	5
4	La arquitectura LPN: encoder, espacio latente y decoder	7
4.1	El Encoder	7
4.2	El Decoder	8
4.3	La Optimización en el Espacio Latente: el corazón de LPN	8
5	Entrenamiento de LPN	11
5.1	La estrategia “leave-one-out”	11
5.2	La función de pérdida	11
5.3	El truco de la reparametrización	12
5.4	Un detalle práctico: <i>stop-gradient</i>	12
6	Experimentos: ¿funciona esto en la práctica?	14
6.1	La tarea “Pattern”: un laboratorio simple	14
6.2	Tarea de manipulación de strings	14
6.3	Generalización fuera de distribución (OOD)	15
6.4	Escalando el tamaño de la especificación	15
6.5	El experimento grande: ARC-AGI 2024	15
7	Conexiones con otras ideas del campo	17
8	Conclusiones, limitaciones y hacia dónde va esto	18
8.1	Lo que el paper afirma haber demostrado	18
8.2	Limitaciones reconocidas por los autores	18
9	Glosario rápido	19

1 Antes de empezar: el mapa del paper

Antes de entrar en detalles técnicos, conviene tener un mapa mental de *qué* está tratando de resolver este paper y *por qué* es interesante. Así, cuando lleguemos a las ecuaciones, sabrás para qué sirven en lugar de verlas como símbolos sueltos.

La pregunta central del paper

Imagina que le muestras a alguien tres ejemplos de una transformación: una entrada y una salida, otra entrada y otra salida, y una tercera. Esa persona, sin que se lo digas explícitamente, “adivina” la regla y la aplica a una entrada nueva. Esto es exactamente lo que hacemos los humanos en el test ARC-AGI (que veremos más abajo) y es lo que el paper quiere que haga una red neuronal: **inferir una “regla” (un programa implícito) a partir de pocos ejemplos, y aplicarla a un caso nuevo, ajustándose (“buscando”) en el momento de la prueba si su primera intuición no alcanza.**

El paper propone una arquitectura llamada **LPN (Latent Program Network)** que combina dos mundos que históricamente estaban separados:

- **Síntesis de programas simbólica (inductiva):** muy buena para generalizar porque produce un programa explícito y verificable, pero el espacio de búsqueda (todas las combinaciones posibles de instrucciones) es gigantesco y normalmente necesita un lenguaje de dominio específico (DSL) diseñado a mano.
- **Aprendizaje profundo (transductivo / in-context learning):** muy escalable porque una red neuronal puede entrenarse con muchísimos datos y no necesita un DSL, pero típicamente no tiene una forma estructurada de “pensar más” en el momento de la prueba: hace una pasada hacia adelante y ya, o necesita reentrenar los pesos completos (costoso y propenso a sobreajustar).

¿Por qué te debería importar esto?

Esta tensión entre “sistemas simbólicos interpretables pero difíciles de escalar” y “redes neuronales escalables pero poco estructuradas” es una de las discusiones más importantes en la investigación actual de IA (por ejemplo, en el debate sobre si los LLMs realmente “razonan” o solo interpolan patrones de entrenamiento). LPN es un ejemplo concreto de una arquitectura que intenta quedarse con lo mejor de ambos mundos: usa una red neuronal (escalable, sin DSL) pero le agrega una capacidad explícita de *búsqueda* en el momento de la prueba (como un sistema simbólico). Entender este paper te da vocabulario e intuiciones que se repiten en: meta-aprendizaje (MAML), autoencoders variacionales (VAE), fine-tuning en tiempo de prueba (test-time training), y benchmarks de razonamiento abstracto como ARC-AGI.

Para pensar

Antes de seguir leyendo: ¿se te ocurre alguna situación de tu vida diaria en la que primero das una respuesta “intuitiva” y rápida a un problema, y luego, si no te convence, te tomas un momento para “buscar” una mejor respuesta pensando más deliberadamente? Guarda esa idea: es exactamente la analogía (Sistema 1 / Sistema 2 de Kahneman) que el paper usa para describir su propia arquitectura.

2 Introducción: el problema y la motivación

2.1 El objetivo de fondo: inteligencia general

El paper arranca con una observación muy citada en el campo: los sistemas de IA actuales pueden llegar a ser sobrehumanos en tareas específicas (ajedrez, Go, StarCraft, predicción de estructura de proteínas), pero **fallan en generalizar más allá de la distribución con la que fueron entrenados** o en adaptarse a situaciones genuinamente nuevas.

Definición: Generalización fuera de distribución (OOD)

Se dice que un sistema generaliza *out-of-distribution* (OOD, fuera de distribución) cuando es capaz de resolver correctamente ejemplos que son distintos —en algún sentido relevante— a los ejemplos que vio durante el entrenamiento. Lo opuesto es la *memorización*: el sistema funciona bien solo porque ya vio algo muy parecido antes.

Los autores retoman la idea de François Chollet (*On the Measure of Intelligence*, 2019): medir solo “habilidades” (¿qué tan bien juega ajedrez un sistema?) no mide inteligencia general, porque esas habilidades se pueden lograr con conocimiento previo masivo o con entrenamiento gigantesco, sin que el sistema realmente haya “aprendido a aprender”. Chollet propone en cambio medir la **eficiencia de adquisición de habilidades**: qué tan bien puede un sistema resolver una tarea *completamente nueva* con muy poca experiencia.

Definición: ARC-AGI

El **Abstraction and Reasoning Corpus (ARC-AGI)** es un benchmark diseñado para medir exactamente esa eficiencia de adquisición de habilidades, controlando el conocimiento previo. Cada tarea consiste en unos pocos pares de *entrada-salida* representados como grillas 2D (de hasta 30x30 celdas, cada celda con uno de 10 colores posibles). El sistema debe inferir la transformación implícita a partir de esos pocos ejemplos y aplicarla correctamente a una nueva grilla de entrada. Es como un test de CI visual para máquinas (y también se le puede tomar a humanos).

¿Por qué grillas de colores?

Las tareas de ARC-AGI están diseñadas para requerir “conocimiento núcleo” (*Core Knowledge*) muy básico —cosas como contar objetos, simetría, completar patrones, mover formas— que casi cualquier humano adquiere de forma natural, sin entrenamiento especializado. La idea es que si un sistema realmente “entiende” estos conceptos elementales, debería poder combinarlos de maneras nuevas para resolver tareas que nunca vio, tal como lo hace un humano viendo el puzzle por primera vez.

2.2 Dos familias de métodos: inductivo vs. transductivo

El paper organiza el trabajo previo en dos grandes familias:

1. **Métodos inductivos**: generan un *programa explícito* (código, en algún lenguaje) que reproduce las transformaciones observadas. Ventaja: si encontramos el programa correcto, generaliza perfectamente y es interpretable. Desventaja: el espacio de todos los programas posibles es combinatoriamente enorme, así que en la práctica se necesita un **DSL** (*Domain Specific Language*, un lenguaje de programación reducido y diseñado a mano para la tarea) que limite las opciones. Diseñar un buen DSL requiere trabajo humano experto y no escala bien a nuevos dominios.
2. **Métodos transductivos (in-context learning)**: en lugar de producir un programa explícito, entrenan una red neuronal para que, dado el conjunto de ejemplos y una nueva entrada, prediga directamente la salida —el “programa” queda implícito en las activaciones internas de la red.

Ventaja: no necesitan DSL, son muy escalables. Desventaja: tienden a ser inconsistentes con la especificación (pueden fallar en reproducir exactamente los ejemplos que se les dieron) y no tienen una forma natural de “pensar más” cuando la tarea es difícil, salvo reentrenar (fine-tunear) los pesos completos en el momento de la prueba —lo cual es costoso computacionalmente y tiende a sobreajustar con tan pocos datos.

Definición: Test-Time Training (TTT)

Es la estrategia de, dado un problema nuevo en el momento de evaluación (no durante el entrenamiento original), realizar pasos adicionales de descenso de gradiente sobre los **parámetros del modelo** usando los pocos ejemplos disponibles de esa tarea específica, antes de generar la predicción final. Es una forma de “especializar” momentáneamente al modelo para el problema que tiene enfrente. El costo es alto porque implica actualizar (potencialmente) millones o miles de millones de parámetros.

LPN nace como una respuesta a esta disyuntiva: **¿se puede tener adaptación en el momento de la prueba (como TTT) sin pagar el costo de actualizar todos los parámetros de la red, y sin necesitar un DSL diseñado a mano?** La apuesta del paper es: sí, si en lugar de buscar en el espacio de *parámetros* de la red (que tiene millones de dimensiones) o en el espacio de *programas de un DSL* (combinatorialmente enorme), buscamos en un **espacio latente continuo y compacto** aprendido por la propia red.

Resumen de la sección

El paper identifica una tensión: los métodos inductivos generalizan bien pero no escalan sin un DSL hecho a mano; los métodos transductivos escalan bien pero no tienen una forma barata de “pensar más” en tiempo de prueba. LPN propone resolver esto buscando, con gradientes, en un espacio latente de programas aprendido —mucho más pequeño que el espacio de parámetros completo de la red, pero suficientemente expresivo como para representar programas.

Para pensar

El texto menciona que el fine-tuning completo en tiempo de prueba (TTT) es “propenso a sobreajustar dado los datos limitados”. ¿Por qué crees que ajustar *millones* de parámetros usando solo 3 o 4 ejemplos de entrada-salida sería una receta para el sobreajuste? Pista: piensa en la relación entre número de parámetros libres y número de puntos de datos disponibles para restringirlos.

3 Fundamentos formales: ¿qué es sintetizar un programa?

Antes de describir la arquitectura, el paper formaliza el problema. Esta sección (Sección 3 del paper, “Background”) puede parecer la más abstracta, pero es la base de todo lo que sigue: si entendés bien esta notación, el resto del paper se lee mucho más fácil.

3.1 Especificación de una tarea

Ecuación 1 — la especificación

$$X^m = \{(x_1^m, y_1^m), \dots, (x_n^m, y_n^m)\}$$

Cómo leerla: X^m es el conjunto de ejemplos (la *especificación*) de la tarea número m . Cada elemento es un par (x_i^m, y_i^m) : una entrada x y su salida correspondiente y , para el i -ésimo ejemplo de esa tarea. En el caso de ARC-AGI, x_i^m y y_i^m son grillas de colores. El superíndice m solo indica “a qué tarea pertenece este conjunto de ejemplos” —no confundir con una potencia.

Definición: Programa que resuelve una tarea

Un programa f (perteneciente al espacio de programas posibles Y) **resuelve** la tarea m si reproduce exactamente todos los ejemplos dados:

$$\forall j \in [1, n], \quad f(x_j^m) = y_j^m$$

Es decir, aplicado a cada entrada de la especificación, produce *exactamente* la salida correspondiente. F_m denota la “función verdadera” que en realidad generó esos pares (la regla oculta que estamos tratando de descubrir).

3.2 El verdadero objetivo: generalizar a un input nuevo

Lo interesante no es solo poder explicar los ejemplos que ya vimos (eso lo puede hacer, en el límite, hasta una tabla de búsqueda memorizada). Lo interesante es poder predecir la salida para una entrada **nueva** que no está en la especificación:

Ecuación 2 — el problema de generalización

$$P_m = \{(x_1^m, y_1^m), \dots, (x_n^m, y_n^m), x_{n+1}^m\}$$

Cómo leerla: es lo mismo que antes, pero agregamos una entrada extra x_{n+1}^m para la cual *no* nos dan la salida. El objetivo del sistema es predecir cuál sería y_{n+1}^m , generalizando la regla implícita en los n ejemplos anteriores.

¿Por qué esto es más difícil de lo que parece?

El paper señala algo sutil y muy importante: si no limitamos la *complejidad* de los programas candidatos, siempre es posible inventar *algún* programa que explique perfectamente los ejemplos dados —aunque ese programa no tenga nada que ver con la regla real que generó los datos (podría ser, literalmente, una gigantesca tabla de casos particulares memorizados). Esta idea está relacionada con la **complejidad de Kolmogorov**: entre todos los programas que explican los datos, preferimos los más simples, porque son los que más probablemente capturan la regla real y por lo tanto generalizan mejor a casos nuevos. Evaluar si un sistema “generalizó de verdad” significa evaluar si predijo bien y_{n+1}^m , no solo si logró explicar los n ejemplos originales.

¿Por qué importa esto?

Esta formulación es, en esencia, un problema de **few-shot learning** (aprendizaje con pocos ejemplos) con la particularidad de que hay una sola tarea a la vez —no hay “clases” compartidas entre tareas como en few-shot learning clásico de clasificación de imágenes. Cada tarea de ARC-AGI es un mini-problema de programación por ejemplos completamente distinto de los demás. Esto es justamente lo que hace tan difícil a ARC-AGI: no alcanza con memorizar patrones estadísticos generales del dataset de entrenamiento, porque cada tarea de evaluación introduce una regla potencialmente nunca vista.

Para pensar

Si le diéramos a un sistema $n = 1$ solo par de entrada-salida, ¿cuántos programas razonables distintos se te ocurre que podrían explicar *ese único ejemplo*? ¿Qué pasa con esa ambigüedad a medida que n crece (tenemos más ejemplos)? Esta pregunta es la semilla de por qué el paper necesita un enfoque *probabilístico* (no determinista) para el encoder, como vas a ver en la próxima sección.

4 La arquitectura LPN: encoder, espacio latente y decoder

Esta es la sección central del paper (Sección 4). LPN separa explícitamente el proceso de inferencia en **tres** componentes, en vez de hacerlo todo con una sola red que va directo de ejemplos a predicción (como hacen los métodos transductivos clásicos).

Encoder $\rightarrow$ Optimización en el espacio latente $\rightarrow$ Decoder

Sistema 1 / Sistema 2, aplicado a una red neuronal

El paper hace una analogía explícita con el libro *Pensar rápido, pensar despacio* de Daniel Kahneman. El **encoder** genera una primera “intuición” rápida sobre cuál podría ser el programa (Sistema 1: rápido, automático). La **optimización en el espacio latente** es un proceso deliberado de búsqueda que refina esa intuición inicial, verificando si realmente explica los datos y corrigiéndola si no (Sistema 2: lento, deliberado, iterativo). El **decoder** finalmente “ejecuta” el programa refinado sobre la entrada nueva.

Antes de continuar, necesitamos una pieza de vocabulario fundamental que el paper da por sabida: los **autoencoders variacionales (VAE)**.

Definición: Espacio latente

Un **espacio latente** es un espacio vectorial de dimensión reducida (por ejemplo, 256 números reales) donde una red neuronal representa de forma comprimida y abstracta la información relevante de una entrada compleja (como una imagen, un texto, o —en este caso— un conjunto de ejemplos entrada-salida). La idea es que puntos cercanos en este espacio representen conceptos “similares”, y que uno pueda moverse por el espacio (interpolando, optimizando, buscando) de forma continua, algo que sería imposible en el espacio discreto de, por ejemplo, código fuente literal.

Definición: Autoencoder Variacional (VAE), en una nuez

Un VAE tiene dos redes: un **encoder** $q_\phi(z \mid \text{datos})$ que, dado un dato de entrada, no produce un único vector latente z , sino los parámetros (media μ y varianza Σ) de una **distribución de probabilidad** sobre posibles z ; y un **decoder** $p_\theta(\text{salida} \mid z)$ que, dado un z muestreado de esa distribución, reconstruye o genera una salida. Se entrena maximizando la verosimilitud de reconstruir bien los datos, mientras se regulariza la distribución latente para que se parezca a una distribución simple conocida (típicamente una normal estándar $\mathcal{N}(0, I)$). Este segundo objetivo se mide con la **divergencia de Kullback-Leibler (KL)**, que cuantifica qué tan distintas son dos distribuciones de probabilidad.

¿Por qué usar algo probabilístico y no un vector fijo?

Podría parecer más simple que el encoder devuelva directamente un único vector z (determinista) en lugar de una distribución. Pero recordá la pregunta de la sección anterior: con pocos ejemplos, existen **muchos** programas plausibles que explican los mismos datos. Un encoder probabilístico modela naturalmente esta **ambigüedad**: en lugar de comprometerse con una única respuesta, describe una región de posibles “programas candidatos”. Esto es exactamente lo que necesita el paso siguiente (la búsqueda), que va a explorar esa región para encontrar el punto que mejor explica los datos.

4.1 El Encoder

El encoder probabilístico $q_\phi(z \mid x, y)$ mapea *cada* par entrada-salida a una distribución (normal multivariada, con media μ y covarianza diagonal Σ) sobre el espacio latente de programas.

Un detalle de diseño importante: LPN procesa **cada par por separado** y luego agrega (promedia) los resultados, en vez de concatenar toda la especificación en una sola secuencia larga (como haría

un Transformer estándar aplicado ingenuamente).

Invariancia a la permutación: un detalle con mucho peso

Si el encoder promedia las representaciones latentes de cada par entrada-salida en lugar de concatenarlos en una secuencia, el resultado **no depende del orden** en que se presentan los ejemplos. Esto tiene sentido: si te muestro los ejemplos de una tarea en el orden 1-2-3 o en el orden 3-1-2, la regla subyacente que estás tratando de inferir es la misma, así que el modelo no debería comportarse distinto. Además, procesar cada par en paralelo (en lugar de secuencialmente) es más eficiente en hardware (GPUs/TPUs) y evita el costo cuadrático de la atención que aparece en Transformers cuando la secuencia de entrada crece (mencionado en la introducción del paper como una limitación del in-context learning).

4.2 El Decoder

El decoder probabilístico $p_\theta(y \mid x, z)$ toma un input x nuevo y un programa latente z , y predice la distribución sobre la salida correspondiente y , **píxel por píxel** (celda por celda de la grilla), sin pasar por ningún DSL intermedio.

Intuición

Aunque la transformación real entrada→salida sea determinista (una regla fija, sin aleatoriedad), el paper igual usa un decoder *probabilístico*. La razón es técnica pero importante: para poder entrenar el sistema completo maximizando una verosimilitud (una probabilidad), es más cómodo tratar toda la cadena como probabilística, incluso si en la práctica, dado el z correcto, la distribución de salida termina siendo muy “puntiaguda” alrededor de la respuesta correcta (casi determinista).

4.3 La Optimización en el Espacio Latente: el corazón de LPN

Este es el componente verdaderamente novedoso del paper. La idea: el encoder da una primera estimación z , pero esa estimación puede no ser lo suficientemente buena —especialmente si la tarea es rara o nunca fue vista en una forma similar durante el entrenamiento. Entonces, LPN **refina** ese z mediante una **búsqueda** activa, usando el propio decoder como “verificador”: prueba distintos z' candidatos y se queda con el que mejor explica (con mayor probabilidad) los ejemplos observados.

Ecuación 3 — el pipeline completo de inferencia

$$\underbrace{z \sim q_\phi(z \mid x, y)}_{\text{Encoder}} \longrightarrow \underbrace{z' = f(p_\theta, z, x, y)}_{\text{Optimización latente}} \longrightarrow \underbrace{\hat{y} \sim p_\theta(y \mid x, z')}_{\text{Decoder}}$$

Cómo leerla, de izquierda a derecha: (1) el encoder muestrea un z inicial a partir de los pares entrada-salida disponibles; (2) una función de búsqueda f toma ese z inicial y, usando el propio decoder p_θ (y los mismos x, y) como “brújula”, produce un z' mejorado; (3) el decoder usa ese z' refinado, junto con la entrada nueva, para predecir la salida $\hat{y}$.

Ecuación 4 — el objetivo de la búsqueda

$$z' \in \arg \max_z \sum_{i=1}^n \log p_\theta(y_i \mid x_i, z)$$

Cómo leerla: buscamos el vector latente z que **maximiza** la suma de las log-verosimilitudes de reconstruir correctamente *todos* los pares de la especificación (n ejemplos). En criollo: “¿qué programa latente, si lo ejecuto (con el decoder) sobre cada input x_i de mis ejemplos, me da como resultado los y_i correctos, en la mayor medida posible?”. Cuanto mejor explique el z

a *todos* los ejemplos simultáneamente, más probable es que capture la regla real (y no una coincidencia).

Definición: Log-verosimilitud (log-likelihood)

$\log p_\theta(y_i | x_i, z)$ es el logaritmo de qué tan probable considera el decoder que y_i sea la salida correcta, dado x_i y el programa latente z . Usamos el logaritmo por conveniencia matemática y numérica: convierte productos de probabilidades (que pueden volverse números extremadamente pequeños) en sumas, que son mucho más estables de calcular y de derivar.

El paper propone (y compara) dos formas concretas de implementar esta búsqueda f :

Opción A — Muestreo (*sampling*). Se generan K candidatos z_k (muestreados de la distribución previa o de la posterior aproximada) y simplemente se elige el que obtiene mayor verosimilitud. Es simple y no requiere que la función sea diferenciable, pero es muy ineficiente: la cantidad de muestras necesarias para tener buena cobertura crece **exponencialmente** con la dimensión del espacio latente (la clásica “maldición de la dimensionalidad”).

Opción B — Ascenso de gradiente (*gradient ascent*). Como el decoder es una red neuronal diferenciable, también lo es su log-verosimilitud respecto de z . Esto permite usar el gradiente para “subir la colina” de verosimilitud de forma mucho más eficiente que probando al azar.

Ecuación 5 — el paso de ascenso de gradiente

$$\forall k \in [1, K], \quad z'_k = z'_{k-1} + \alpha \cdot \nabla_z \sum_{i=1}^n \log p_\theta(y_i | x_i, z) \Big|_{z=z'_{k-1}}$$

Cómo leerla: partiendo de z'_0 (la media de los z que dio el encoder), en cada paso k calculamos el **gradiente** de la verosimilitud total respecto a z , evaluado en el punto actual z'_{k-1} , y damos un pequeño paso en esa dirección, de tamaño α (la *tasa de aprendizaje* o *learning rate* de esta búsqueda). Si α es suficientemente pequeño, cada paso mejora (o al menos no empeora) la verosimilitud. Repetimos esto K veces.

¿Qué es un gradiente, en términos simples?

Pensá en la verosimilitud como la altura de un terreno montañoso, donde cada punto del terreno es un posible vector latente z y la altura representa “qué tan bien ese z explica los datos”. El **gradiente** en un punto te dice, literalmente, hacia dónde subir más rápido desde ahí. El ascenso de gradiente es el proceso de dar pasitos repetidos siempre en la dirección de mayor pendiente ascendente, hasta llegar (idealmente) a una cima —un z' que explica muy bien los datos observados.

¿Por qué esto es más barato que el fine-tuning clásico (TTT)?

Este es uno de los puntos de venta centrales del paper. En TTT, el ascenso/descenso de gradiente se hace sobre *todos los parámetros de la red* (θ , que pueden ser cientos de millones de números). Acá, el ascenso de gradiente se hace únicamente sobre el vector latente z (por ejemplo, 256 números en los experimentos de ARC-AGI). Es una **optimización muchísimo más chica y barata**, aunque igual usa el poder expresivo de toda la red (a través del decoder) para evaluar qué tan bueno es cada candidato. Es buscar en un mapa de baja dimensión guiado por un juez muy sofisticado (el decoder), en vez de reescribir el juez mismo en cada evaluación.

		Espacio de búsqueda	Costo por paso
Síntesis (DSL)	simbólica	Combinaciones discretas de instrucciones (enorme)	Evaluación simbólica, no diferenciable
Test-Time (TTT)	Training	Todos los parámetros θ de la red	Backprop por toda la red completa
LPN (este paper)		Espacio latente continuo z (compacto)	Backprop solo hasta z , reutilizando θ fijo

Para pensar

En la Ecuación 5, el paso de gradiente parte de z'_0 , que es **el promedio de los latentes que da el encoder para cada par entrada-salida por separado** (no un punto aleatorio). ¿Por qué crees que partir de una buena inicialización (en vez de, por ejemplo, un z aleatorio) es tan importante cuando usamos ascenso de gradiente? Pista: pensá en la superficie de verosimilitud como un terreno con múltiples colinas (óptimos locales) — no solo una montaña perfecta.

Para pensar

El paper deja explícitamente como trabajo futuro explorar **estrategias evolutivas** en vez de ascenso de gradiente puro para esta búsqueda. ¿En qué escenarios te imaginás que el ascenso de gradiente podría fallar (por ejemplo, quedarse atascado) y una estrategia evolutiva —que explora de forma más “global” y estocástica— podría ser más robusta?

5 Entrenamiento de LPN

Ya vimos cómo LPN hace *inferencia* (Sección 4 del paper, arriba). Ahora: ¿cómo se entrenan el encoder ϕ y el decoder θ para que todo esto funcione bien desde el principio?

5.1 La estrategia “leave-one-out”

Para simular en entrenamiento la misma situación que se va a enfrentar en la prueba (predecir una salida nueva a partir de las demás), LPN usa una estrategia de tipo *leave-one-out* (“dejar uno afuera”):

La idea, en una frase

Para reconstruir la salida y_i del ejemplo i , el modelo **no tiene permitido mirar el propio par** (x_i, y_i) . Solo puede usar los otros $n - 1$ pares de la misma tarea para construir su latente, y luego debe predecir y_i a partir de x_i y ese latente.

¿Por qué esta restricción es crucial?

Si al encoder se le permitiera ver el propio par (x_i, y_i) que después tiene que reconstruir, el modelo podría hacer **trampa**: en vez de aprender una representación abstracta del “programa” subyacente, le alcanzaría con comprimir literalmente y_i dentro del latente y luego “copiarlo” de vuelta —un atajo (*shortcut*) que no generaliza a nada nuevo. Al forzar al modelo a inferir y_i usando *solo* los otros ejemplos, lo obligamos a aprender la regla general, no a memorizar casos puntuales. Esta es una instancia de un patrón muy común en Machine Learning: diseñar la tarea de entrenamiento para que la única forma de resolverla bien sea aprendiendo la abstracción correcta.

El procedimiento completo, para reconstruir cada y_i :

1. Para cada $j \neq i$, muestrear $z_j \sim q_\phi(z | x_j, y_j)$ con el encoder.
2. Agregarlos promediando: $\bar{z} = \frac{1}{n-1} \sum_{j \neq i} z_j$.
3. Refinar con optimización latente (por ejemplo, ascenso de gradiente) para obtener z'_i .
4. Calcular la log-verosimilitud (en la práctica, *cross-entropy*) de predecir correctamente y_i a partir de x_i y z'_i con el decoder.

5.2 La función de pérdida

Ecuación 6 — las dos pérdidas de entrenamiento

$$\mathcal{L}_{\text{rec}}(\phi, \theta) = \sum_{i=1}^n -\log p_\theta(y_i | x_i, z'_i) \quad \mathcal{L}_{\text{KL}}(\phi) = \sum_{i=1}^n D_{\text{KL}}(q_\phi(z | x_i, y_i) \| \mathcal{N}(0, I))$$

Cómo leerlas:

- $\mathcal{L}_{\text{rec}}$ (*pérdida de reconstrucción*) mide qué tan mal el decoder predice las salidas correctas —cuanto menor, mejor. Es, en la práctica, la **entropía cruzada** (*cross-entropy*) estándar de clasificación, ya que cada píxel de salida se predice como una de 10 clases de color posibles.
- $\mathcal{L}_{\text{KL}}$ (*pérdida de regularización*) mide qué tan lejos está la distribución que produce el encoder de una distribución normal estándar simple $\mathcal{N}(0, I)$ —esto evita que el espacio latente se vuelva un desorden arbitrario y ayuda a que sea “suave” y navegable (bueno para la búsqueda por gradiente posterior).

Definición: Divergencia de Kullback-Leibler (KL)

$D_{KL}(q \parallel p)$ mide qué tan diferente es una distribución de probabilidad q respecto de otra p (no es una distancia simétrica, pero sirve como “penalización de disimilitud”). Cuando q es idéntica a p , $D_{KL} = 0$. Acá se usa para empujar la distribución latente aprendida q_ϕ a parecerse a una normal estándar, lo cual regulariza el espacio latente.

Ambas pérdidas se combinan en un único objetivo total, ponderado por un factor β :

$$\mathcal{L}_{\text{total}}(\phi, \theta) = \mathcal{L}_{\text{rec}}(\phi, \theta) + \beta \mathcal{L}_{\text{KL}}(\phi)$$

El rol de β : una tensión entre dos objetivos

Hay una tensión natural entre ambos términos. Si solo importara $\mathcal{L}_{\text{rec}}$, el encoder podría usar un espacio latente desordenado y complejo (lo que sea que le permita reconstruir mejor). Si β es demasiado alto, el espacio latente se vuelve muy “comprimido” y simple pero pierde capacidad expresiva para representar programas distintos. Este balance —muy estudiado en la literatura de VAEs, en particular en los llamados β -VAE— es un hiperparámetro que hay que ajustar cuidadosamente.

5.3 El truco de la reparametrización

Hay un problema técnico sutil: si el encoder produce una distribución y *muestreamos* un z de ella, el proceso de muestreo en sí mismo no es diferenciable (es una operación aleatoria), lo cual rompe la cadena de backpropagation necesaria para entrenar ϕ .

Definición: Truco de reparametrización (*reparameterization trick*)

En vez de muestrear z directamente de $\mathcal{N}(\mu, \Sigma)$, muestreamos primero ruido de una normal estándar fija, $\epsilon \sim \mathcal{N}(0, I)$ —esta parte sí es aleatoria pero **no depende de los parámetros del encoder**, así que no hace falta derivar a través de ella— y luego construimos z de forma determinista y diferenciable:

$$z = \mu + \epsilon \cdot \Sigma$$

Como μ y Σ son funciones diferenciables de los parámetros ϕ del encoder, ahora sí se puede calcular el gradiente de todo el proceso respecto a ϕ mediante backpropagation estándar.

¿Por qué importa esto?

Este truco (introducido por Kingma & Welling en el paper original de VAE, 2013) es una de las ideas más influyentes del deep learning generativo moderno. Aparece prácticamente en cualquier modelo que combine *muestreo aleatorio* con *entrenamiento por gradiente*: VAEs, ciertos modelos de difusión, políticas estocásticas en aprendizaje por refuerzo, etc. Vale la pena que te quede grabada esta idea porque la vas a volver a encontrar constantemente.

5.4 Un detalle práctico: *stop-gradient*

El paper menciona que se puede elegir si el gradiente de la pérdida de reconstrucción fluye “hacia atrás” a través de todo el proceso de optimización latente (es decir, derivar respecto a θ considerando cómo z'_i cambiaría si θ cambiara) o si se aplica un **stop-gradient** sobre la actualización del propio z durante el ascenso de gradiente, para ahorrar cómputo.

Definición: Stop-gradient

Un **stop-gradient** es una operación que, en el forward pass, no hace nada (deja pasar el valor tal cual), pero en el backward pass “corta” el flujo del gradiente en ese punto, tratando ese valor como una constante. Se usa cuando queremos usar un valor calculado dinámicamente pero no queremos pagar el costo (en memoria y cómputo) de derivar a través de todo el proceso

que lo generó.

Sorprendentemente, los experimentos del paper (Tabla 1) muestran que usar *stop-gradient* no solo es más barato computacionalmente, sino que además **mejora** el desempeño en tiempo de prueba —un resultado que vale la pena recordar: a veces menos señal de gradiente (más “ruidosa” o menos completa) puede actuar como una forma de regularización implícita.

Para pensar

La pérdida de reconstrucción $\mathcal{L}_{\text{rec}}$ depende de ϕ (los parámetros del encoder) de forma indirecta, a través del muestreo de z_j . ¿Por qué el truco de reparametrización hace posible que el gradiente de $\mathcal{L}_{\text{rec}}$ respecto a ϕ se pueda calcular de forma estándar con backpropagation, cuando sin este truco no se podría? Intenta explicarlo con tus propias palabras antes de volver a leer la definición de arriba.

6 Experimentos: ¿funciona esto en la práctica?

La Sección 5 del paper presenta una batería de experimentos diseñados no solo para mostrar “LPN gana”, sino para entender **por qué y bajo qué condiciones** funciona. Vale la pena prestar atención al diseño experimental tanto como a los números.

6.1 La tarea “Pattern”: un laboratorio simple

Antes de lanzarse a ARC-AGI (costoso de entrenar), los autores diseñan una tarea sintética mucho más simple para poder iterar rápido y entender la dinámica del modelo: grillas de 10x10 con un píxel azul que indica dónde pegar un patrón de 4x4 específico de cada “programa”.

¿Por qué empezar con una tarea de juguete?

Es una práctica muy común (y muy sabia) en investigación: antes de gastar días de cómputo en TPUs entrenando en un benchmark difícil y caro (ARC-AGI), primero se valida la idea central en un entorno pequeño, rápido de entrenar y fácil de analizar/visualizar. Si la idea no funciona ni siquiera en el caso simple, es una señal temprana de alarma barata.

Resultado clave (Tabla 1 del paper): entrenar con *cero* pasos de ascenso de gradiente (“Grad 0”; equivalente a un modelo puramente amortizado, sin búsqueda) da resultados pobres incluso cuando se le permite mucho cómputo de búsqueda después. En cambio, entrenar **con** al menos un paso de ascenso de gradiente (“Grad 1”) y luego evaluar con 100 pasos de gradiente en tiempo de prueba alcanza **99.5%** de exactitud, contra apenas 67.5% del modelo entrenado sin gradiente. Esto es una idea muy importante:

La idea más importante del paper, quizás

Para que la búsqueda en tiempo de prueba funcione bien, el espacio latente tiene que haber sido entrenado “sabiendo” que después se va a buscar en él. No alcanza con entrenar un buen modelo amortizado (rápido, sin búsqueda) y después, como una idea de último momento, agregarle búsqueda por encima. El espacio latente que resulta de entrenar *con* optimización latente durante el entrenamiento tiene una geometría distinta —más “navegable” por gradiente— que el que resulta de entrenar sin ella. Esto se llama a veces un **sesgo inductivo**: una elección de diseño en el entrenamiento que predispone al modelo hacia soluciones que se comportan bien bajo el procedimiento de inferencia que realmente se va a usar.

También comparan muestreo vs. ascenso de gradiente como método de búsqueda: el ascenso de gradiente gana ampliamente, confirmando la intuición de la Sección 4 sobre la ineficiencia del muestreo en espacios de más de pocas dimensiones.

6.2 Tarea de manipulación de strings

Para verificar que las conclusiones no son un artefacto de la estructura espacial de las grillas, replican el análisis en una tarea sintética de secuencias de números (con más de 100 millones de programas posibles, compuestos por reglas parametrizadas encadenadas). Aquí aparece un resultado muy revelador comparando LPN contra TTT (Tabla 2):

- TTT **empeora** a medida que se le da más cómputo de ajuste en tiempo de prueba (de 78.1% con 0 pasos a 11.8% con 100 pasos) — es decir, sobreajusta a los pocos ejemplos disponibles al seguir actualizando todos sus parámetros.
- LPN, en cambio, **mejora o se mantiene estable** con más pasos de búsqueda (de 75.6% a 81.3% en la variante Grad 0 de entrenamiento), sin mostrar ese colapso.

¿Por qué LPN resiste mejor el sobreajuste que TTT?

Piensa en la cantidad de “grados de libertad” que cada método ajusta en tiempo de prueba: TTT ajusta millones de parámetros de la red completa usando 3-5 ejemplos —un problema masivamente subdeterminado, donde hay muchísimas formas de “memorizar” esos pocos ejemplos sin capturar la regla real. LPN ajusta solo un vector latente de tamaño mucho menor (cientos de dimensiones, no millones), y ese vector está restringido además a vivir dentro de una región razonable del espacio latente gracias a la regularización KL del entrenamiento. Menos grados de libertad para ajustar en tiempo de prueba, con una regularización de por medio, se traduce en menos margen para sobreajustar.

6.3 Generalización fuera de distribución (OOD)

Este es, para muchos lectores, el resultado más contundente del paper (Tabla 3): en la tarea Pattern evaluada en configuraciones OOD (patrones nunca vistos durante el entrenamiento), tanto in-context learning como TTT obtienen básicamente **0% de exactitud**, incluso con 100 pasos de ajuste. LPN, entrenado con un paso de gradiente y evaluado con 100 pasos de búsqueda, recupera **88%** de exactitud.

¿Por qué importa esto?

Este resultado es la prueba más directa de la tesis central del paper: que tener una capacidad de *búsqueda estructurada* en tiempo de prueba —no solo más cómputo, sino cómputo dirigido de forma inteligente por un espacio latente bien entrenado— es lo que permite adaptarse a situaciones genuinamente nuevas, algo que ni el forward-pass puro (in-context learning) ni el fine-tuning de fuerza bruta (TTT) logran en este experimento.

6.4 Escalando el tamaño de la especificación

¿Qué pasa si le damos al modelo más o menos ejemplos de los que vio durante entrenamiento? (Figura 3 del paper.) In-context learning **sobreajusta al tamaño de especificación con el que fue entrenado** y no generaliza bien a otros tamaños. TTT necesita bastantes ejemplos y mucho cómputo para funcionar razonablemente. LPN, en cambio, generaliza suavemente: funciona razonablemente incluso con menos ejemplos de los vistos en entrenamiento, y sigue mejorando con más.

Intuición

Esto se conecta con una limitación estructural de los métodos transductivos basados en concatenar ejemplos (in-context learning): si el modelo se entrenó viendo secuencias de, digamos, 5 pares de entrada-salida concatenados, tiene poco incentivo (o capacidad, según la arquitectura) para comportarse bien frente a una secuencia de 15 pares. En cambio, como LPN **promedia** las representaciones latentes de cada par (Sección 4.1), agregar más pares es simplemente promediar sobre más elementos: una operación que naturalmente se comporta bien sin importar cuántos pares haya.

6.5 El experimento grande: ARC-AGI 2024

Finalmente, entrenan un modelo LPN de **178 millones de parámetros** con un espacio latente de 256 dimensiones, durante 100 mil pasos (2 días en una TPU v4-32), usando el dataset **re-arc** (generado procedualmente para parecerse en distribución al set de entrenamiento oficial de ARC-AGI, sin incluirlo directamente).

Definición: FLOPs

FLOPs (*floating point operations*) es una medida de la cantidad total de operaciones de punto flotante (sumas, multiplicaciones) que un cómputo realiza. Se usa como una forma de comparar de manera relativamente justa el “presupuesto de cómputo” entre distintos métodos en tiempo de prueba (más pasos de búsqueda o de fine-tuning = más FLOPs usados), independientemente de detalles de implementación de hardware.

Resultados clave (Tabla 4 del paper):

- **Dentro de distribución:** LPN supera claramente a TTT en todos los niveles de cómputo probados (por ejemplo, 80.00% vs. 58.50% con 2×10^{13} FLOPs).
- **Fuera de distribución:** LPN **duplica su desempeño** al activar la búsqueda en tiempo de prueba, escalando el cómputo de 2×10^{11} a 2×10^{15} FLOPs. Sin embargo, en los niveles más altos de cómputo, TTT termina superando levemente a LPN.

¿Por qué TTT alcanza a LPN con muchísimo cómputo OOD?

Los propios autores ofrecen una hipótesis honesta: LPN fue entrenado con una diversidad de programas relativamente limitada (el dataset re-arc, aunque grande, no cubre toda la variedad de *Core Knowledge* que exige el set de evaluación). Esto restringe qué tan expresivo y “suave” puede llegar a ser el espacio latente aprendido para representar programas verdaderamente novedosos. TTT, al ajustar directamente todos los parámetros de la red (un espacio de búsqueda mucho más grande y flexible, aunque más costoso y con más riesgo de sobreajuste), puede en el límite de mucho cómputo alcanzar soluciones que el espacio latente comprimido de LPN no logra representar bien. Este es un ejemplo honesto de un **trade-off**: un espacio de búsqueda más chico y estructurado es más eficiente en general, pero puede toparse con un techo de expresividad si no fue entrenado con suficiente diversidad.

También es interesante notar que ambos métodos (LPN y TTT) superan a dos enfoques previos basados en LLMs mucho más grandes: CodeIt (220M parámetros, usando CodeT5 preentrenado) y un enfoque con **text-davinci** (175 mil millones de parámetros) — mostrando que, al menos en este benchmark, una arquitectura bien diseñada con búsqueda estructurada puede superar a modelos varios órdenes de magnitud más grandes sin esa estructura.

Para pensar

El paper menciona que, al momento de escribirse, el modelo de mejor desempeño en ARC-AGI (**o3-preview**) alcanzaba 75.7% pero a un costo de \$200 por tarea. LPN, mucho más barato de entrenar y de usar en inferencia, queda por debajo de eso en desempeño absoluto. ¿Qué preguntas te parece que deberíamos hacernos para comparar de forma justa dos sistemas que difieren tanto en escala de cómputo/costo como en desempeño absoluto? ¿Tiene sentido comparar solo el “score” final, o hace falta normalizar por algún tipo de presupuesto (cómputo, dinero, datos de entrenamiento)?

Para pensar

La Sección 5.7 menciona que LPN logra *componer* programas vistos durante el entrenamiento en combinaciones nuevas durante la búsqueda en tiempo de prueba, aunque de forma no del todo robusta (hay casos de éxito y casos de fallo). ¿Por qué crees que la capacidad de **componer** piezas de conocimiento previamente aprendidas de formas nuevas es considerada, en general, uno de los sellos distintivos de un razonamiento verdaderamente “inteligente” y no solo memorístico?

7 Conexiones con otras ideas del campo

La Sección 2 (Related Work) del paper conecta LPN con varias líneas de investigación previas. Vale la pena conocerlas aunque sea de forma somera, porque son puntos de referencia muy usados en el campo.

- **MAML (Model-Agnostic Meta-Learning, Finn et al. 2017)**: un método de meta-aprendizaje donde se hacen actualizaciones de gradiente sobre *todos* los parámetros de la red en un “bucle interno” rápido, para adaptarse a una tarea específica con pocos datos. LPN comparte la filosofía de “optimizar en tiempo de prueba/adaptación”, pero limita esa optimización a un espacio latente mucho más chico en vez de todos los parámetros.
- **LEO (Rusu et al. 2018)**: muy cercano a LPN en espíritu —introduce un cuello de botella latente y hace la optimización del bucle interno en ese espacio latente en vez de en los parámetros completos—, pero usa una **hipernetwork** (una red que genera los pesos de otra red) para decodificar el latente en parámetros. LPN evita esa hipernetwork: en cambio, aprovecha directamente las capacidades de “in-context conditioning” de un Transformer para condicionar la generación en el vector latente.
- **Inferencia variacional semi-amortizada (SVI, Kim et al. 2018)**: el paper explícitamente interpreta a LPN bajo este marco. Primero se hace *inferencia amortizada* (el encoder, entrenado sobre todo el dataset, da una respuesta rápida para cualquier input nuevo sin optimización adicional). Luego, para cada caso particular, se hacen pasos extra de optimización para achicar el “*amortization gap*” —la diferencia entre lo que el encoder amortizado logra y lo que sería óptimo para ese caso puntual. La optimización en el espacio latente de LPN es exactamente ese paso adicional de ajuste fino semi-amortizado.
- **Task vectors en LLMs (Hendel et al. 2023)**: trabajos recientes muestran que los LLMs, al hacer in-context learning, parecen desarrollar internamente representaciones parecidas a “vectores de tarea” en sus activaciones. LPN puede verse como una forma de **forzar explícitamente** esa separación entre “la tarea/programa” y el resto del procesamiento, en lugar de esperar que emerja implícitamente y de forma desordenada dentro de las activaciones de un modelo entrenado con otro objetivo.

Definición: Amortized inference / inferencia amortizada

Se llama así a la estrategia de entrenar una red (el encoder) para resolver de una sola pasada (rápida) un problema de inferencia que, sin esa red, requeriría resolver una optimización costosa desde cero para cada nuevo caso. El “costo” de resolver muchas instancias del problema de optimización se **amortiza** (se paga una sola vez) durante el entrenamiento de la red, y luego cada instancia nueva se resuelve barato en tiempo de inferencia.

Para pensar

Ahora que conocés el concepto de *amortization gap*: en tus propias palabras, ¿por qué la optimización en el espacio latente de LPN puede describirse como “achicar la brecha de amortización” en vez de simplemente llamarla “una búsqueda más”? ¿Qué le agrega ese marco teórico a tu comprensión de por qué la arquitectura funciona?

8 Conclusiones, limitaciones y hacia dónde va esto

8.1 Lo que el paper afirma haber demostrado

1. Una nueva arquitectura (LPN) que integra adaptación en tiempo de prueba *directamente en su diseño*, aprendiendo un espacio latente de programas y buscando en él, en vez de dejar la adaptación como un truco de post-procesamiento.
2. Evidencia de que **entrenar con conciencia de la búsqueda posterior** (usando ascenso de gradiente durante el entrenamiento, no solo en la prueba) produce espacios latentes mucho más “navegables” y útiles para la adaptación en tiempo de prueba.
3. Evidencia de que LPN generaliza a tamaños de especificación (cantidad de ejemplos) mayores a los vistos en entrenamiento, algo que los métodos in-context tradicionales no logran sin fine-tuning adicional.

8.2 Limitaciones reconocidas por los autores

- **Diversidad limitada de programas de entrenamiento:** el espacio de programas al que LPN estuvo expuesto durante el entrenamiento (incluso usando aumentación de datos) es relativamente angosto, lo que limita qué tan expresivo puede volverse el espacio latente aprendido —una posible explicación de por qué TTT lo alcanza con presupuestos de cómputo muy altos en el escenario OOD de ARC-AGI.
- **Búsqueda limitada a optimizadores estándar de gradiente:** no se exploran métodos de búsqueda alternativos como estrategias evolutivas, que podrían balancear mejor exploración (buscar ampliamente) y explotación (refinar localmente).
- **Representaciones continuas, no discretas:** al no usar representaciones discretas de programas, la capacidad de **composición** sistemática (combinar piezas de programas de forma exacta y modular, como haría un lenguaje de programación real) queda limitada —los propios autores muestran ejemplos donde la composición en tiempo de prueba funciona y ejemplos donde falla.

La honestidad científica como parte del método

Vale la pena notar, como lección general de cómo leer papers, que los autores no ocultan los casos donde su método *no* gana (por ejemplo, TTT superando a LPN con cómputo extremadamente alto en el escenario OOD de ARC-AGI). Reportar y tratar de explicar los resultados negativos o mixtos —en vez de solo mostrar los números favorables— es parte de lo que separa un paper riguroso de una simple demostración de marketing.

Pregunta integradora final

Repasando todo lo que vimos: si tuvieras que explicarle la idea central de LPN a alguien en **una sola oración**, sin usar la palabra “latente” ni “gradiente”, ¿cómo la dirías? Intentalo antes de seguir leyendo esta guía de nuevo — es un buen test de si realmente interiorizaste la idea o solo reconocés el vocabulario técnico.

Pregunta integradora final

El paper describe a LPN como un punto intermedio entre síntesis simbólica de programas y aprendizaje profundo puramente transductivo. ¿En qué otras áreas de la IA (que conozcas o se te ocurran) ves una tensión parecida entre “sistemas estructurados/interpretables pero difíciles de escalar” y “sistemas de aprendizaje profundo escalables pero menos estructurados”? ¿Se te ocurre cómo se podría aplicar una idea similar a la de LPN (una búsqueda estructurada en un espacio comprimido aprendido) en ese otro dominio?

9 Glosario rápido

Término	Significado en este paper
Síntesis de programas	Generar/inferir un programa que reproduzca ejemplos de entrada-salida dados.
DSL (<i>Domain-Specific Language</i>)	Lenguaje de programación reducido, diseñado a mano, que acota el espacio de programas posibles.
Inductivo	Enfoque que produce un programa explícito y verificable.
Transductivo / in-context learning	Enfoque que predice la salida directamente, sin programa explícito intermedio.
Test-Time Training (TTT)	Ajustar los parámetros completos de la red en el momento de la prueba, usando los pocos ejemplos disponibles.
Espacio latente	Espacio vectorial comprimido donde se representan conceptos abstractos (acá, “programas”).
VAE (Autoencoder Variacional)	Arquitectura encoder-decoder que aprende una distribución latente regularizada, en vez de un único vector fijo.
Divergencia KL	Medida de qué tan distintas son dos distribuciones de probabilidad; se usa para regularizar el espacio latente.
Reparameterization trick	Técnica para poder derivar a través de un proceso de muestreo aleatorio.
Ascenso de gradiente	Método iterativo que sigue la dirección de mayor pendiente positiva para maximizar una función.
Inferencia amortizada	Resolver de una sola pasada (rápida) un problema que normalmente requeriría optimización costosa desde cero.
Amortization gap	Diferencia entre lo que logra la inferencia amortizada y lo que sería óptimo para un caso particular.
FLOPs	Unidad para medir la cantidad de cómputo (operaciones de punto flotante) usada por un método.
ARC-AGI	Benchmark de razonamiento abstracto basado en grillas de colores, diseñado para medir eficiencia de adquisición de habilidades.
Generalización OOD	Capacidad de resolver correctamente casos distintos a los vistos en entrenamiento.

Sugerencia para seguir estudiando

Una buena forma de consolidar lo aprendido es intentar responder, sin mirar el documento, las preguntas “Para pensar” de cada sección, y luego comparar tus respuestas con el texto correspondiente. Si querés profundizar más antes de avanzar a los apéndices del paper original (que incluyen detalles de arquitectura, hiperparámetros, y visualizaciones adicionales del espacio latente), los conceptos previos más útiles para reforzar son: autoencoders variacionales (VAE), la divergencia KL, y meta-aprendizaje (MAML).